

Laravel 12 + JWT Full Project Guide

This document contains the complete Laravel 12 backend setup for the Student Teacher Portal project using JWT Authentication, MySQL Database, and custom AppModel architecture.

Step 1 - Create Laravel Project

```
composer create-project laravel/laravel student-teacher-api
cd student-teacher-api
```

Step 2 - Install JWT Package

```
composer require php-open-source-saver/jwt-auth
```

Step 3 - Publish JWT Config

```
php artisan vendor:publish --provider="PHPOpenSourceSaver\JWTAuth\Providers\LaravelServiceProvider"
```

Step 4 - Generate JWT Secret

```
php artisan jwt:secret
```

Step 5 - Run Laravel Server

```
php artisan serve
```

Step 6 - Database .env Setup

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=student_teacher_db
DB_USERNAME=root
DB_PASSWORD=
CACHE_STORE=file
SESSION_DRIVER=file
QUEUE_CONNECTION=sync
```

Step 7 - tbl_users SQL

```
CREATE TABLE tbl_users (
    intId BIGINT PRIMARY KEY AUTO_INCREMENT,
    vch_name VARCHAR(100),
    vch_email VARCHAR(150) UNIQUE,
    vch_password VARCHAR(255),
    int_role SMALLINT NOT NULL DEFAULT 2,
    tinStatus TINYINT DEFAULT 1,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Step 8 - tbl_students SQL

```
CREATE TABLE tbl_students (
    intStudentId BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id BIGINT,
```

```
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    );
```

Step 9 - tbl_teachers SQL

```
CREATE TABLE tbl_teachers (
    intTeacherId BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id BIGINT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Step 10 - AppModel.php

```
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Support\Facades\DB;

class AppModel extends Model
{
    public $timestamps = false;

    public function getAll()
    {
        return DB::table($this->table)->get();
    }

    public function store($data)
    {
        return DB::table($this->table)->insertGetId($data);
    }
}
```

Step 11 - User.php

```
<?php
namespace App\Models;
use Illuminate\Foundation\Auth\User as Authenticatable;
use PHPOpenSourceSaver\JWTAuth\Contracts\JWTSubject;

class User extends Authenticatable implements JWTSubject
{
    protected $table = 'tbl_users';
    protected $primaryKey = 'intId';
    public $timestamps = false;

    protected $fillable = [
        'vch_name',
        'vch_email',
        'vch_password',
        'int_role'
    ];

    protected $hidden = [
        'vch_password'
    ];

    public function getAuthPassword()
    {
        return $this->vch_password;
    }
}
```

```

    }

    public function getJWTIdentifier()
    {
        return $this->getKey();
    }

    public function getJWTCustomClaims()
    {
        return [];
    }
}

```

Step 12 - StudentModel.php

```

<?php
namespace App\Models;

class StudentModel extends AppModel
{
    protected $table = 'tbl_students';
    protected $primaryKey = 'intStudentId';
}

```

Step 13 - TeacherModel.php

```

<?php
namespace App\Models;

class TeacherModel extends AppModel
{
    protected $table = 'tbl_teachers';
    protected $primaryKey = 'intTeacherId';
}

```

Step 14 - Create AuthController

```
php artisan make:controller Api/AuthController
```

Step 14 (b) - AuthController.php

```

<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;

use Illuminate\Http\Request;

use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Validator;
use Illuminate\Support\Facades\Hash;

use App\Models\User;
use App\Models\StudentModel;
use App\Models\TeacherModel;

use PHPOpenSourceSaver\JWTAuth\Facades\JWTAuth;

class AuthController extends Controller

```

```

{
    protected $studentModel;
    protected $teacherModel;

    public function __construct()
    {
        $this->studentModel = new StudentModel();
        $this->teacherModel = new TeacherModel();
    }

    public function register(Request $request)
    {
        $validator = Validator::make($request->all(), [

            'vch_name'      => 'required',
            'vch_email'     => 'required|email|unique:tbl_users,vch_email',
            'vch_password'  => 'required|min:6',
            'int_role'      => 'required'

        ]);

        if ($validator->fails()) {

            return response()->json([
                'status' => false,
                'errors' => $validator->errors()
            ]);
        }

        DB::beginTransaction();

        try {

            $userId = DB::table('tbl_users')->insertGetId([

                'vch_name'      => $request->vch_name,

                'vch_email'     => $request->vch_email,

                'vch_password'  => Hash::make($request->vch_password),

                'int_role'      => $request->int_role,

                'created_at'    => now()
            ]);

            /*
            role:
            1 = teacher
            2 = student
            */

            if ($request->int_role == 1) {

                $this->teacherModel->store([

                    'user_id'    => $userId,

                    'created_at' => now()
                ]);
            }
        }
    }
}

```

```

        if ($request->int_role == 2) {

            $this->studentModel->store([

                'user_id'    => $userId,

                'created_at' => now()
            ]);

        }

        DB::commit();

        return response()->json([

            'status' => true,

            'message' => 'Registration Successful'
        ]);

    } catch (\Exception $e) {

        DB::rollback();

        return response()->json([

            'status' => false,

            'message' => $e->getMessage()
        ]);

    }

}

/* public function login(Request $request)
{
    $credentials = [

        'vch_email' => $request->vch_email,

        // 'vch_password' => $request->vch_password
        'password' => $request->vch_password
    ];
    $token = JWTAuth::attempt($credentials);
    // dd($token);
    if (!$token) {

        return response()->json([

            'status' => false,

            'message' => 'Invalid Email or Password'
        ]);

    }

    $user = JWTAuth::user();

    return response()->json([

        'status' => true,

        'message' => 'Login Successful',

        'token' => $token,

```

```

        'user' => $user
    });
} */

public function login(Request $request)
{
    $credentials = [
        'vch_email' => $request->vch_email,
        'password' => $request->vch_password // 'password' key is required by Laravel Auth
    ];

    // DEBUG: Check if user exists and what key it returns
    /* $user = \App\Models\User::where('vch_email', $request->vch_email)->first();

    dd([
        'user_found'    => $user ? true : false,
        'user_data'     => $user,
        'primary_key'   => $user ? $user->getKey() : 'NO USER',
        'intId_value'   => $user ? $user->intId : 'NO USER',
        'jwt_subject'   => $user ? $user->getJWTSubject() : 'NO USER',
    ]); */

    $token = JWTAuth::attempt($credentials);

    if (!$token) {
        return response()->json([
            'status' => false,
            'message' => 'Invalid Email or Password'
        ]);
    }

    $user = JWTAuth::user();

    return response()->json([
        'status' => true,
        'message' => 'Login Successful',
        'jwt_token' => $token,
        'user' => $user
    ]);
}

public function logout()
{
    try {

        JWTAuth::invalidate(JWTAuth::getToken());

        return response()->json([
            'status' => true,
            'message' => 'Logout Successful'
        ]);

    } catch (\Exception $e) {

        return response()->json([

```

```

        'status' => false,

        'message' => $e->getMessage()
    ]);
    }
}

public function profile()
{
    try {

        $user = JWTAuth::parseToken()->authenticate();

        // dd($user);

        return response()->json([

            'status' => true,

            'user' => $user
        ]);

    } catch (\Exception $e) {

        return response()->json([

            'status' => false,

            'message' => $e->getMessage()
        ]);

    }
}
}

```

Step 15 - routes/api.php

```

<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\AuthController;

Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

Route::middleware('auth:api')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::get('/profile', [AuthController::class, 'profile']);
});

```